

API Connector

for Odoo 18

Outbound REST, JSON-RPC, GraphQL and SOAP calls from Odoo

Version 18.0.1.2.0

IROKOO

sbt@irokoo.fr | https://www.irokoo.fr

Version History

Version	Date	Changes
18.0.1.0.0	2026-10	Documentation in English and French, screenshots of version 18
18.0.1.2.0	2026-10	Variables as lines, detected and created on save, also applied to headers; request body built from lines; variables read on an Odoo field; request preview

Table of Contents

1 Overview 4
1.1 What the module does 4
1.2 Features 4
1.3 Module sheet 4
1.4 Menus 4
2 Installation 5
2.1 Prerequisites 5
2.2 Install the module 5
2.3 What the installation adds 5
2.4 Upgrading from version 18.0.1.0.0 5
3 Getting Started: a First Call 5
4 Collections 6
4.1 Collection Settings 6
4.2 Headers and Variables 7
5 Endpoints 8
5.1 Basic Configuration 8
5.2 Headers 9
5.3 Request Body 9
5.4 Response and Statistics 12
6 Authentication 12
6.1 Inherit or define 12
6.2 Methods 12
6.3 OAuth 2.0 13
7 Variables 13
7.1 Variables tab 13
7.2 Value of a variable 13
7.3 Test Record 13
8 Request Preview 14
9 Automatic Execution 14
9.1 From a scheduled action 14
9.2 Reading the result 16
10 Monitoring 16
10.1 Call log 16
10.2 Retention 16
11 Export and Import 16
11.1 Export 16
11.2 Import 17
11.3 Postman collections 18

12 Access Rights 19
13 Technical Reference 19
13.1 Models 19
13.2 Useful methods 19
13.3 Variables as JSON 20
13.4 Deleting a critical endpoint 20

1 Overview

1.1 What the module does

API Connector lets Odoo call external APIs. Each call is described in a form, the way Postman does it: address, HTTP method, authentication, headers, request body. Run it with one click to test it, then hand it to a scheduled action or a server action so that it runs on its own. Every call leaves a trace in a log.

The module is the outbound counterpart of Advanced JSON-RPC API, which exposes Odoo data to inbound calls.

1.2 Features

Feature	What it does
Collections	Group the endpoints of one API and share the base URL, authentication, headers and variables.
Endpoints	One configured call: API type, method, path, headers, body, timeout, SSL verification.
API types	REST, JSON-RPC, GraphQL, SOAP/XML.
Authentication	None, Basic Auth, Bearer Token, API Key in a header, OAuth 2.0 (client credentials).
Variables	`{name}` placeholders replaced in the URL, the headers and the body. Entered on the collection or the endpoint, or read on a field of the record sent.
Built body	JSON or form body assembled from key / variable lines, with a type per value.
Preview	Method, URL, headers and body as they would be sent, before any call.
Execution	Button in the form or in the list, and call from code in a scheduled action or a server action.
Monitoring	Log of every call (status, duration, data sent and received), success rate per endpoint and per collection.
Export and import	Configuration as JSON, and import of Postman collections v2.0 to v2.2.

1.3 Module sheet

Property	Value
Technical name	`iro_api_connector`
Version	18.0.1.2.0
Odoo version	18
License	OPL-1
Author	IROKOO
Odoo dependencies	`base`, `web`
Python libraries	`requests`, `jsonschema`

1.4 Menus

The module adds the **API Connector** application:

Menu	Entry	Content
Configuration	Collections	Collections and their shared settings
	Endpoints	All configured calls
	Import Configuration	Import of a JSON export or of a Postman collection
Monitoring	Call Logs	Call history, filtered on the last 7 days by default

2 Installation

2.1 Prerequisites

The Python libraries `requests` and `jsonschema` must be available in the Odoo environment. They are listed in the module's `requirements.txt`:

```
pip install -r iro_api_connector/requirements.txt
```

2.2 Install the module

1. Copy the `iro_api_connector` folder into a directory of the addons path (`addons_path`).
2. Enable developer mode, then **Apps > Update Apps List**.
3. Search for **API Connector** and click **Activate**.

2.3 What the installation adds

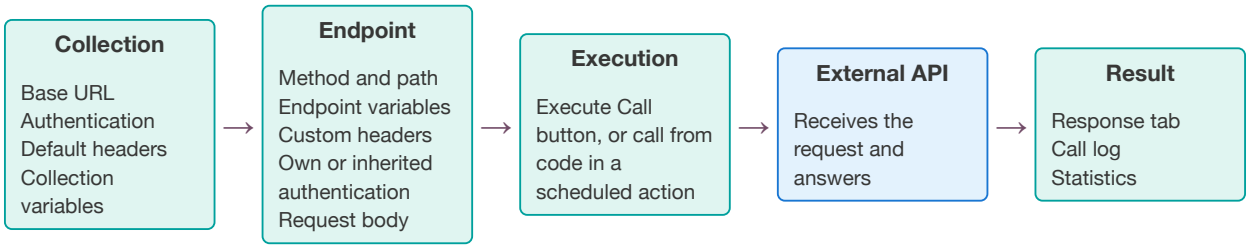
- The **API Connector** application and its menus.
- Sample collections and endpoints calling public APIs (JSONPlaceholder, httpbin.org, api-adresse.data.gouv.fr, Rick and Morty GraphQL). They serve as templates and can be archived.
- A daily scheduled action, **API Connector: Cleanup Old Logs**, which deletes log entries older than 30 days.

2.4 Upgrading from version 18.0.1.0.0

Updating the module turns the variables entered as JSON on collections and endpoints into variable lines. The addresses called do not change. The JSON fields stay readable and writable from code (see Technical Reference).

3 Getting Started: a First Call

The diagram below sums up what a call is made of.



The steps below create a collection pointing at httpbin.org, a public service that echoes back what it receives: the response shows exactly the headers and body sent by Odoo.

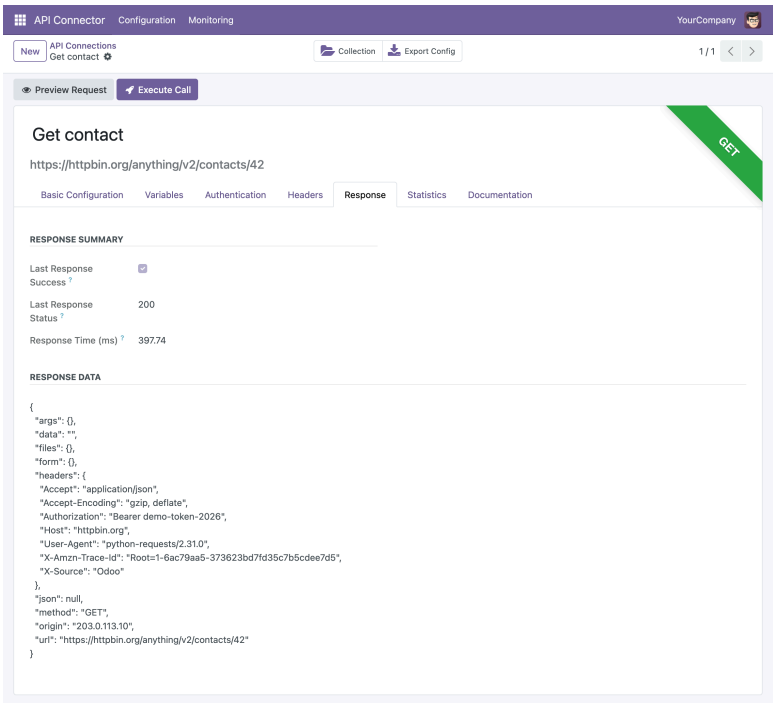


Figure 1: httpbin.org response: the `Authorization` header comes from the collection, the URL received both variables.

1. **API Connector > Configuration > Collections > New**. Name: `Partner API (demo)`, Base URL: `https://httpbin.org`.
2. **Authentication** tab: **Bearer Token**, token `demo-token-2026`.
3. **Headers and Variables** tab: default headers `{"Accept": "application/json"}`, then a collection variable `api_version` with the value `v2`. Save.
4. **Add Endpoint** button. Name: `Get contact`, method `GET`, path `/anything/{api_version}/contacts/{contact_id}`. Save: the variable `contact_id` is created, empty, in the **Variables** tab. Give it the value `42` and save. The full URL shown under the name becomes `https://httpbin.org/anything/v2/contacts/42`.
5. Click **Execute Call**, then open the **Response** tab.

The call also appears in **Monitoring > Call Logs**, and the counters of the **Statistics** tab went up by one.

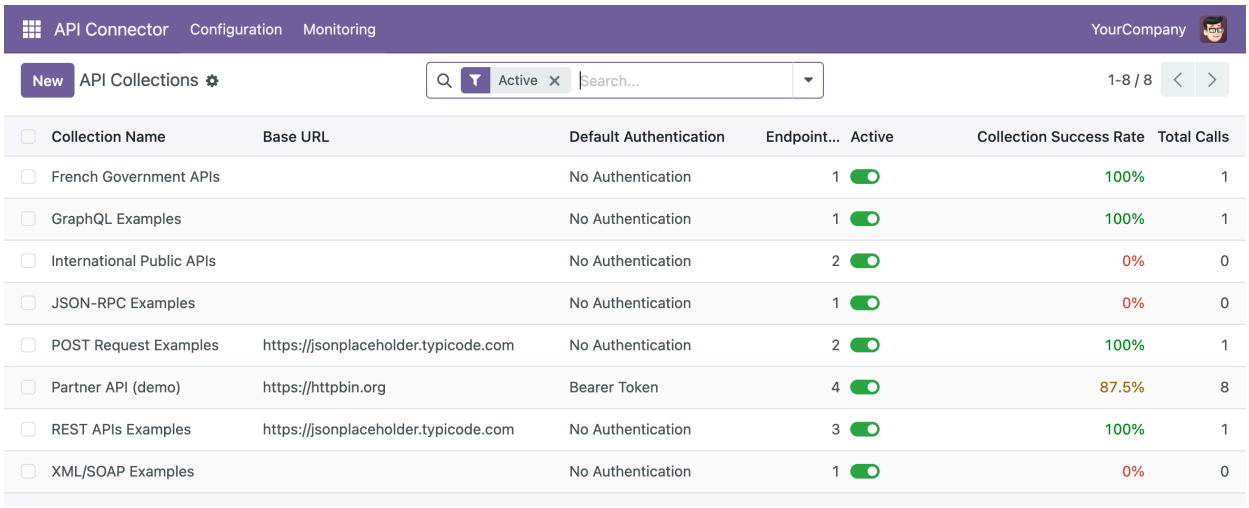
4 Collections

A collection groups the endpoints of one API. Its settings are the defaults of all its endpoints.

4.1 Collection Settings

Field	Purpose
Collection Name	Required.
Description	Free text under the name.
Base URL	Common beginning of the addresses, for example `https://api.example.com`. Used by the endpoints that tick **Use Collection URL**.
Default Authentication	Used by the endpoints whose authentication is **Inherit from Collection**. Details in the Authentication chapter.
Total Calls, success rate	Sum of the counters of the collection's endpoints.

Form buttons:



Collection Name	Base URL	Default Authentication	Endpoint...	Active	Collection Success Rate	Total Calls
French Government APIs		No Authentication	1		100%	1
GraphQL Examples		No Authentication	1		100%	1
International Public APIs		No Authentication	2		0%	0
JSON-RPC Examples		No Authentication	1		0%	0
POST Request Examples	https://jsonplaceholder.typicode.com	No Authentication	2		100%	1
Partner API (demo)	https://httpbin.org	Bearer Token	4		87.5%	8
REST APIs Examples	https://jsonplaceholder.typicode.com	No Authentication	3		100%	1
XML/SOAP Examples		No Authentication	1		0%	0

Figure 2: Collection list: base URL, default authentication, number of endpoints, success rate and total calls.

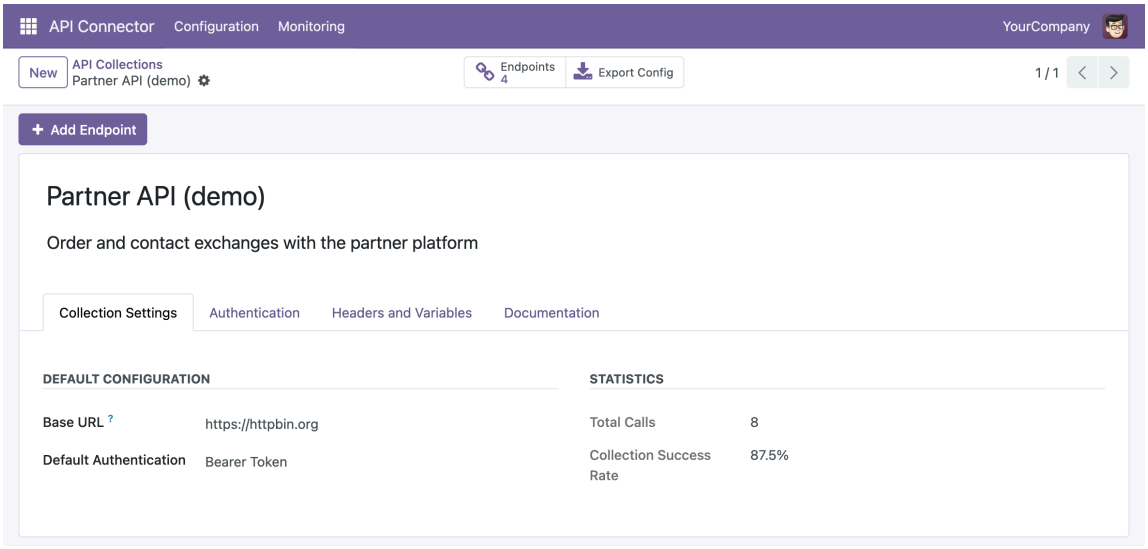


Figure 3: Collection form, Collection Settings tab.

- **Add Endpoint** opens an endpoint prefilled with the base URL, authentication and default headers of the collection.
- **Endpoints** shows the list of the collection's endpoints.
- **Export Config** downloads the collection and its endpoints as JSON (see the Export and Import chapter).

Archiving a collection archives all its endpoints. Unarchiving it reactivates them.

4.2 Headers and Variables

- **Default Headers**: JSON object sent with every call of the collection's endpoints. It must be valid JSON: saving is refused otherwise. It may contain `{name}` placeholders.
- **Collection Variables**: one line per variable (name, value, description), shared by all the endpoints of the collection.

When the collection is saved, the `{name}` placeholders of its base URL and default headers that match no variable are created, empty, in this list.

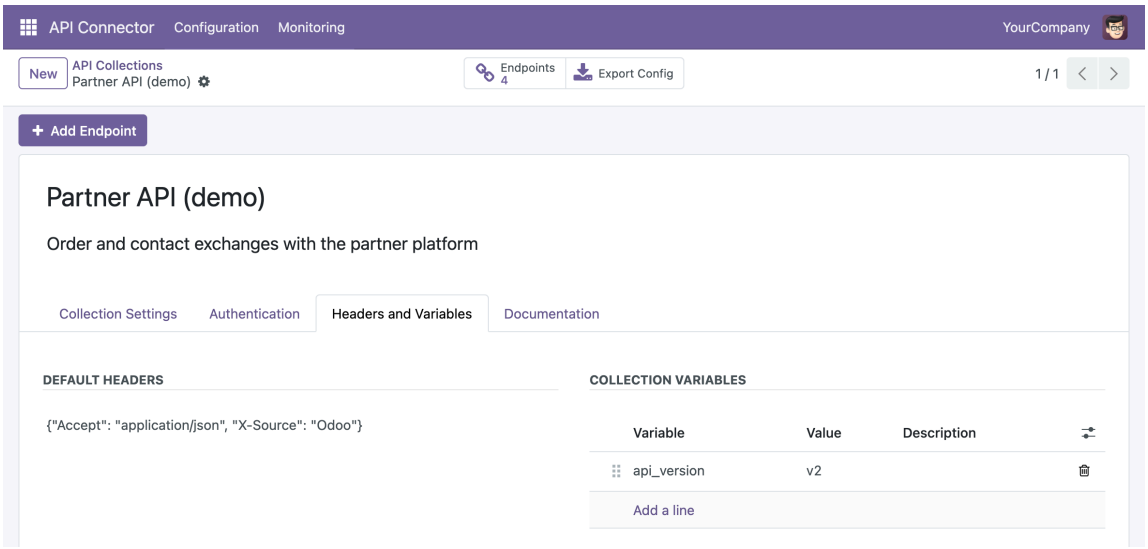


Figure 4: Headers and Variables tab: default headers as JSON, variables as lines.

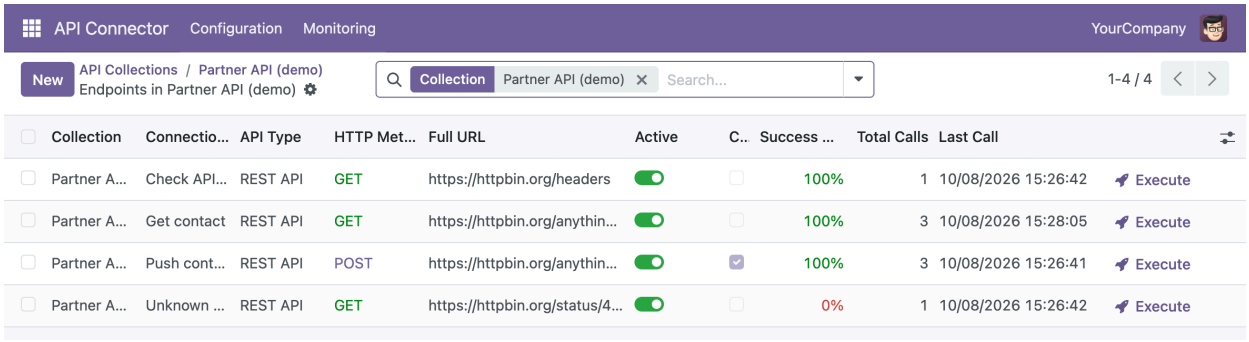


Figure 5: Endpoints of a collection. The C. (critical) column flags the ones protected against deletion.

5 Endpoints

An endpoint describes a call. The list shows them all, with the method in colour, the success rate and an **Execute** button on each line.

5.1 Basic Configuration

Field	Purpose
Collection	Parent collection, optional.
API Type	REST, JSON-RPC, GraphQL or SOAP/XML. For JSON-RPC, GraphQL and SOAP/XML the method switches to POST and can no longer be changed. JSON-RPC also sets the body to JSON.
HTTP Method	GET, POST, PUT, PATCH or DELETE, editable in REST.
Use Collection URL	Shown once a collection is set. Ticked, the base URL comes from the collection. Unticked, the endpoint has its own base URL.
Base URL	Must start with `http://` or `https://`.
Endpoint Path	Appended to the base URL. May contain `{name}` placeholders (see the Variables chapter).
Timeout (seconds)	From 1 to 300, 30 by default.
Verify SSL Certificate	Ticked by default.
Expected Response Type	JSON, XML, plain text or binary. Sets how the response is read (see Response).

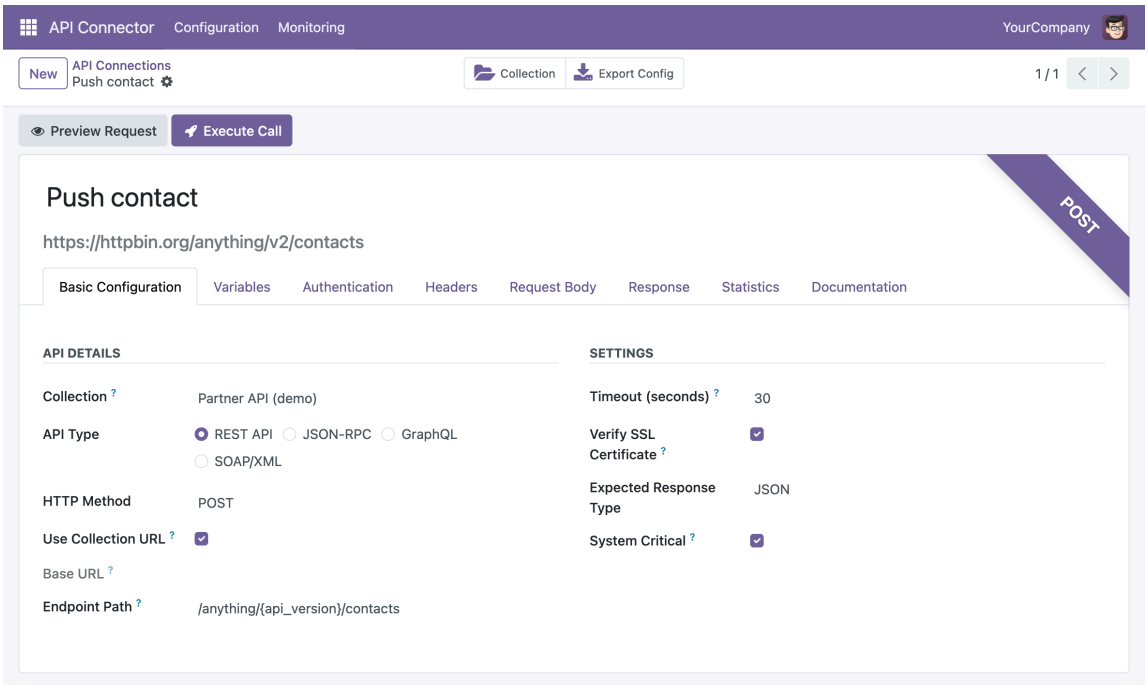


Figure 6: Basic Configuration tab. The ribbon shows the method, the full URL is computed under the name.

System Critical	Prevents deletion of the endpoint. It can still be disabled with the Active field.

The **Full URL** is the base URL followed by the path, once the variables that have a value are replaced. It is recomputed on every change. At execution time, the address is computed again with the data passed to the call and the record sent.

5.2 Headers

The **Headers** tab takes a JSON object of custom headers. When sending, Odoo builds the headers in this order, each step winning over the previous one:

1. default headers of the collection;
2. custom headers of the endpoint;
3. replacement of the `{name}` placeholders in the values;
4. authentication header (`Authorization` or the API key header);
5. `Content-Type` derived from the body type, only when not already given: `application/json`, `application/xml`, `application/x-www-form-urlencoded` or `application/octet-stream`.

5.3 Request Body

The tab is hidden for a REST call in GET or DELETE. For a JSON or form body, **Body Mode** chooses between **Written**, the text typed, and **Built from lines**.

5.3.1 Written body

Body type	Sent
No Body	No body.
JSON	The text entered, checked as valid JSON at call time.
Form Data (urlencoded)	The key/value pairs of the **Form Data** field, entered as a JSON object.
Form Data (multipart)	The key/value pairs of the **Form Data** field, encoded as `x-www-form-urlencoded` like the previous type.

XML, Raw Text	The text entered, as is.
Binary (base64)	The base64 content entered, decoded before sending.

The `{name}` placeholders of the body and of the form data are replaced by the variables (see the Variables chapter).

5.3.2 Body built from lines

Column	Purpose
Key	Name of the value in the body. In JSON, a dotted key builds nested objects: `customer.address.city` gives `{"customer": {"address": {"city": ...}}}`. In form data, the key stays as is.

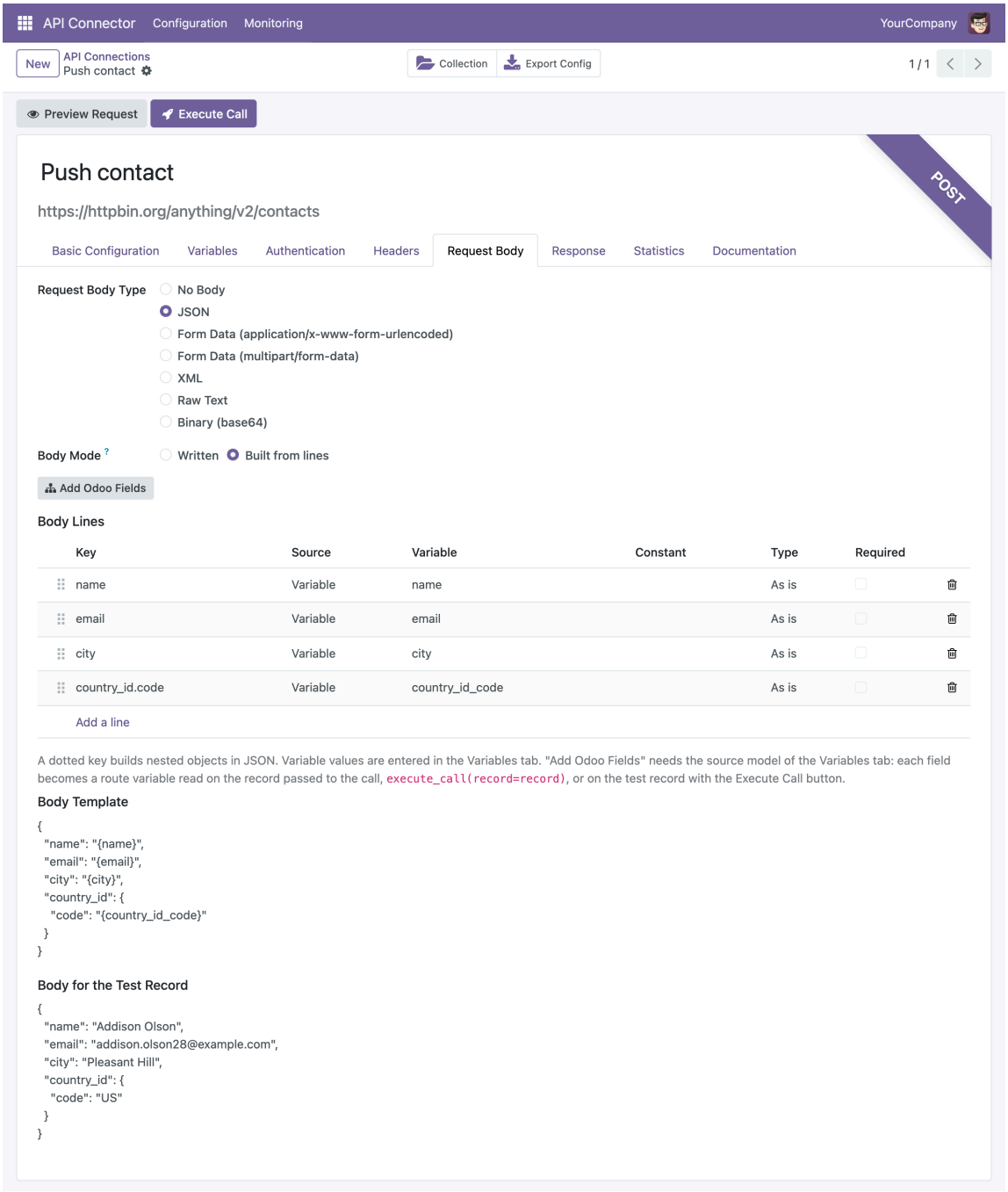


Figure 7: Body built from lines, with its template and the actual body for the test record.

Source	**Variable** (value of the named variable) or **Constant** (value entered).
Type	**As is** keeps the value received, **Text**, **Integer**, **Decimal** or **Boolean** convert it (`yes`, `true`, `1` for true).
Required	An empty value blocks the call. The error names the key.

An empty value that is not required gives `null` in JSON and is left out of form data. **Body Template** shows the body with `{variables}` in place of the values, **Body for the Test Record** the actual body (see Test Record).

5.3.3 Adding Odoo fields

The **Add Odoo Fields** button needs a source model (**Variables** tab). It opens the field tree of that model, browsed like the one of the standard export: a click unfolds a relation, down to three levels, **+** or a double click adds the field.

Add to the Body creates, for each field, a route variable reading that field (`country_id.code` gives `{country_id_code}`) and a body line using it, of type **As is**: a many2one gives its ID, a one2many or many2many the list of IDs (or of values when the path goes on, for instance `order_line.product_id.default_code`), a date the ISO format, an empty field `null`. The body switches to **Built from lines**.

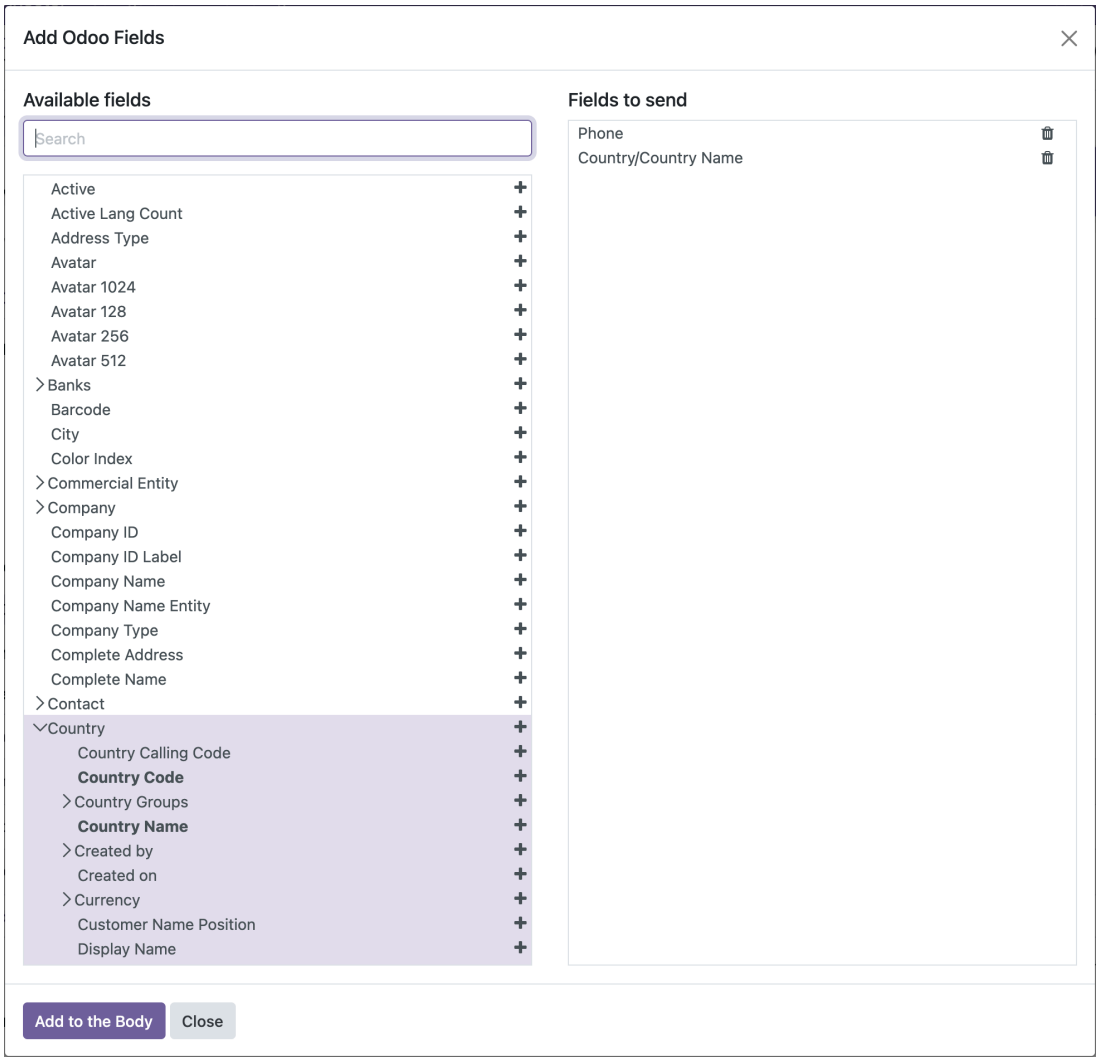


Figure 8: Choosing the fields to send. Relations unfold as in the standard export.

5.4 Response and Statistics

The **Response** tab shows the result of the last call: success, HTTP status, response time in milliseconds and data received. A JSON response is shown indented. With the **Binary** expected response type, the data is kept as base64.

A call succeeds when the HTTP status is below 400. A 4xx or 5xx status, a timeout or a connection error count as a failure. On a connection error or a timeout no data is received: the error message is read in the log, **Error Details** tab.

The **Statistics** tab gives the total calls, the successful calls, the success rate and the date of the last call.

6 Authentication

6.1 Inherit or define

An endpoint uses **Inherit from Collection** by default: it then uses its collection's authentication. It can also define its own, which takes precedence.

6.2 Methods

Method	Fields	Sent
No Authentication		Nothing.
Basic Auth	Username, password	HTTP Basic authentication.
Bearer Token	Bearer Token / API Key	`Authorization: Bearer <token>`
API Key	API Key Header Name (`X-API-Key` by default), Bearer Token / API Key	`<header name>: <key>`
OAuth 2.0	Client ID, client secret, token URL, scope	`Authorization: Bearer <access token>`

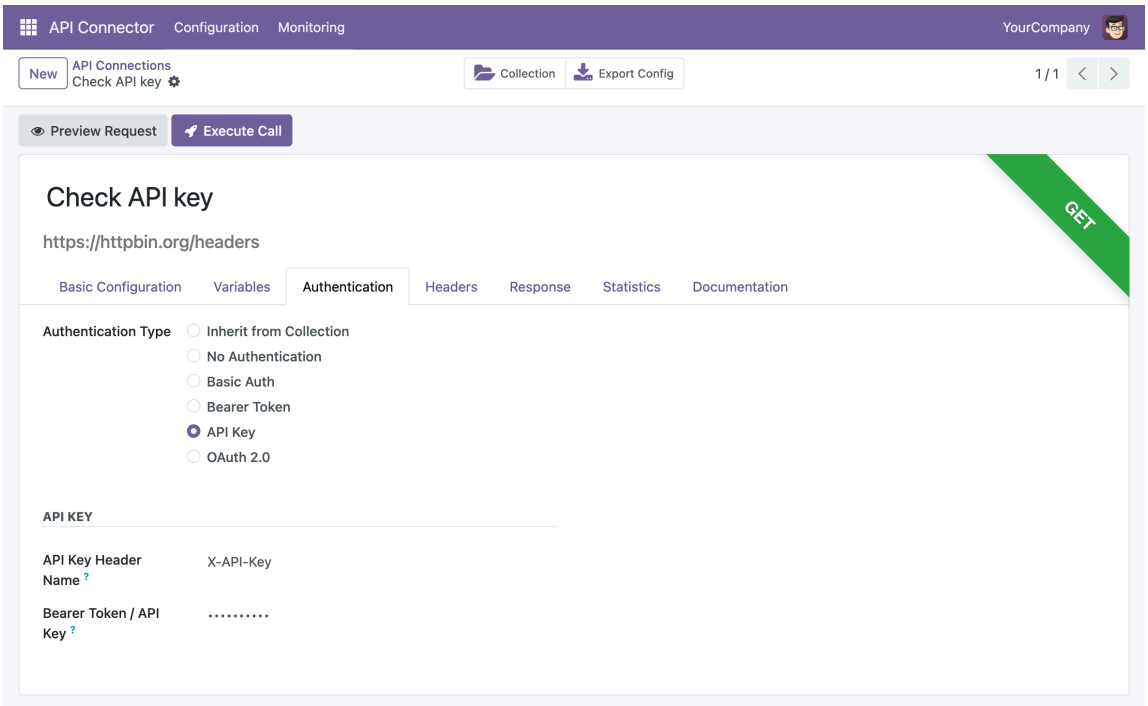


Figure 9: An endpoint replacing its collection's Bearer token with an API key.

Password, token and secret fields are masked on screen.

6.3 OAuth 2.0

The **Get New Access Token** button requests a token from the token URL with the **client credentials** flow (`grant_type=client_credentials`, with the client ID, secret and scope). The access token, refresh token and expiry date are stored, and a notification shows the result.

- On a collection, the token obtained is also used by the endpoints that inherit the authentication.
- The module does not renew the token when it expires: click the button again, or paste a token into the field provided under the tokens.

7 Variables

A placeholder is written `{name}` (letters, digits, _, `.` and `-`, starting with a letter or _). It is replaced in the URL, the headers and the request body.

7.1 Variables tab

Block	Content
Odoo Record	**Source Model**: model of the records sent, whose fields can feed the variables. **Test Record**: a record of that model, for trials.
Detected Variables	Each `{name}` found, where it is used, where it is defined and the value that would be sent. A highlighted line has no value.
Route Variables	Name, **Odoo Field** (picked with the standard field selector), value and description.
Collection Variables	Reminder, read only, of the collection's variables.

When an endpoint is saved, the `{name}` placeholders of its URL, headers and body that are defined neither on it nor on its collection are created, empty, in **Route Variables**.

7.2 Value of a variable

Rank	Source
1	Data passed to the call: `execute_call({'name': value})`.
2	Value entered on the route variable.
3	Odoo field of the route variable, read on the record passed to the call.
4	Value of the collection variable.

The first source giving a value wins. A variable without value leaves its `{name}` as is in the request.

```
connector.execute_call({'contact_id': '42'})   # value passed to the call
connector.execute_call(record=partner)         # fields read on the contact
```

7.3 Test Record

Variables reading an Odoo field need a record. From code, it is the one passed to the call. From the form, it is the **Test Record**: the **Execute Call** button uses it, the detected variables table shows the values read on it, and **Body for the Test Record** shows the body that would be sent. It is only used for trials.

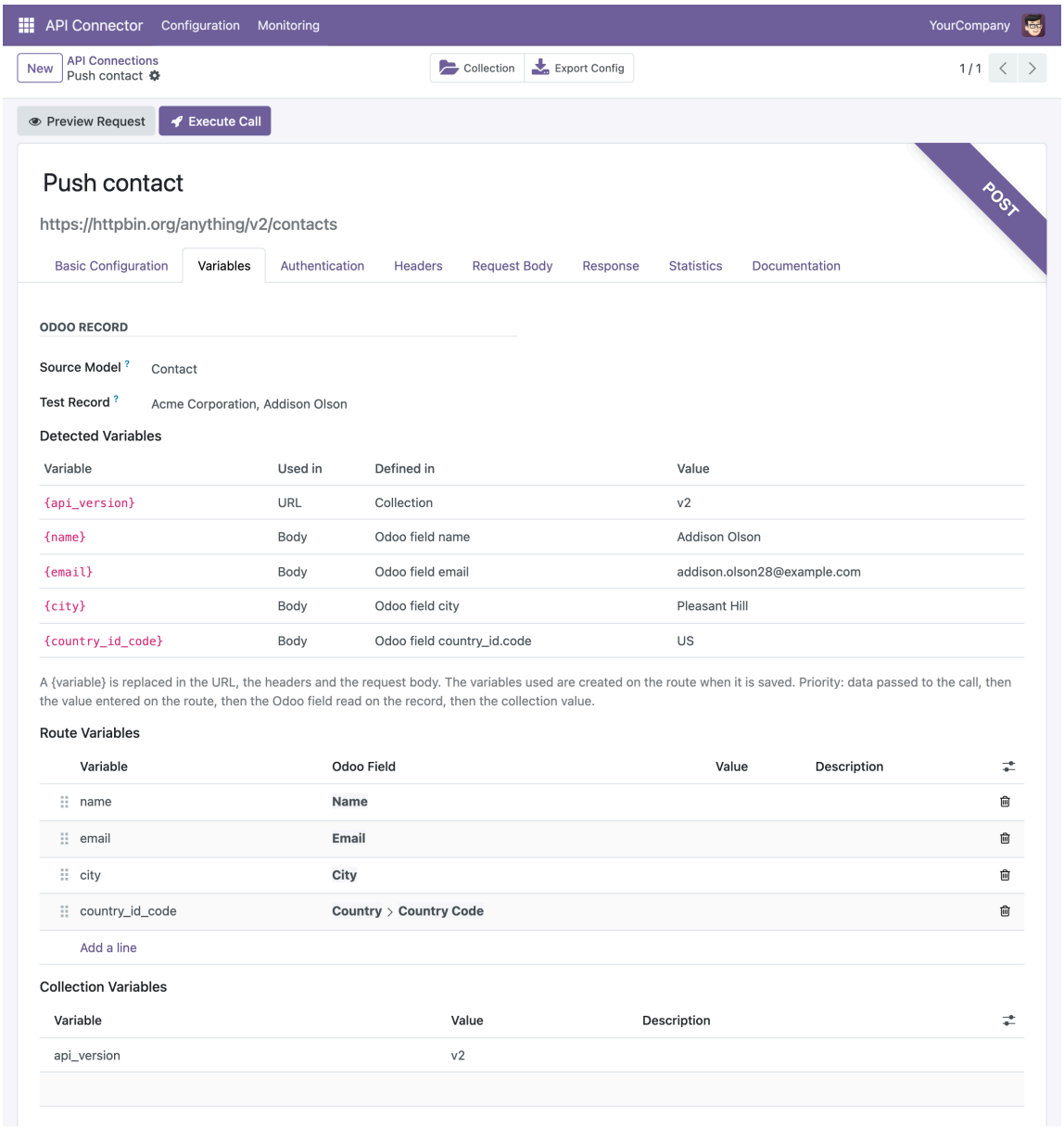


Figure 10: Variables tab: source model, detected variables with their value, route variables.

8 Request Preview

The **Preview Request** button opens the list of detected variables, each with the value the call would use now and its current source. A value changed in the list immediately recomputes the method, the URL, the headers and the body. Nothing is saved on the endpoint and nothing is sent.

A cleared value leaves its `{name}` as is. An error, for instance an empty required value, is shown in red. Authentication headers (`Authorization`, `X-API-Key`…) are truncated.

9 Automatic Execution

9.1 From a scheduled action

The code of a scheduled action or a server action calls `execute_call` on the endpoint, passing the record to send, the variable values, or both.

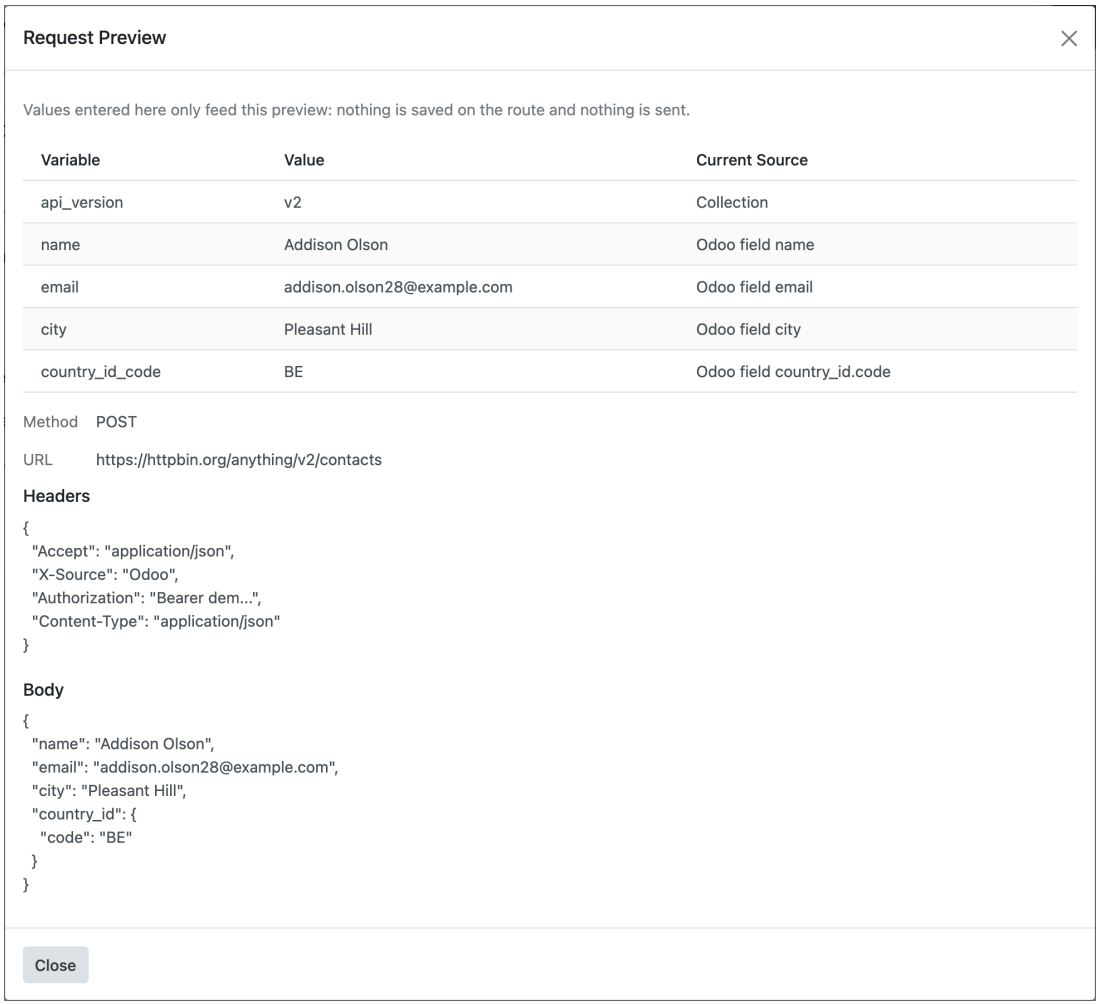


Figure 11: Request preview: the value entered for the country code goes into the URL and the body.

```
connector = env['api.connector'].browse(17)
partners = env['res.partner'].search([('email', '!=', False)], limit=3)
```

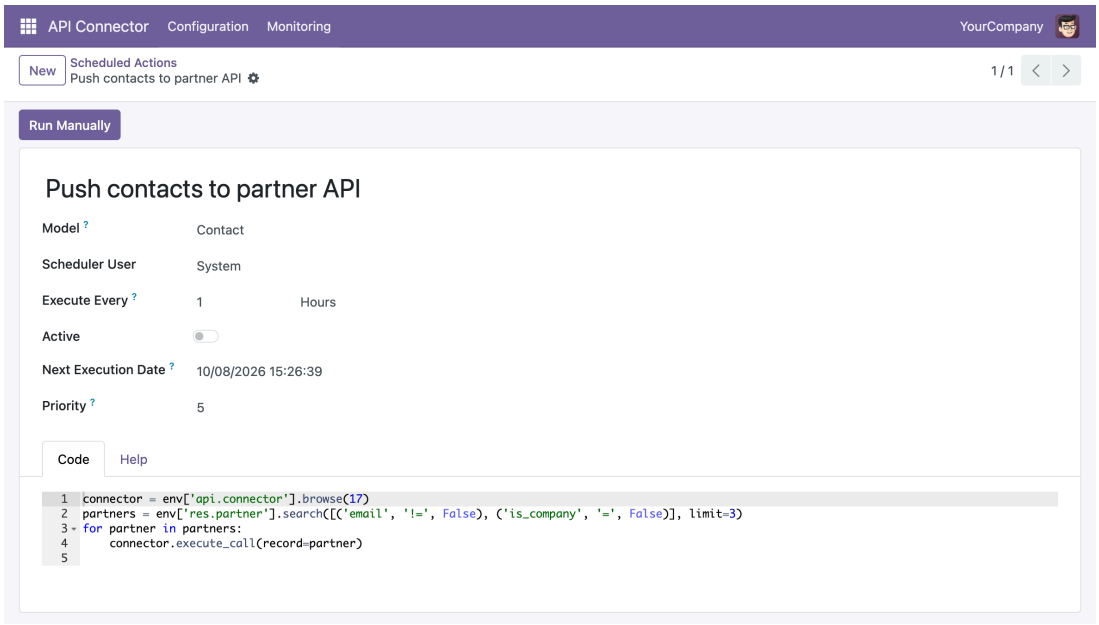


Figure 12: Scheduled action sending three contacts: one call per contact, each with its own values.

```
for partner in partners:
    connector.execute_call(record=partner)
```

The endpoint variables `name`, `email` and `city` read the contact's fields. Without a source model, pass the values: `connector.execute_call({'name': partner.name})`. The endpoint ID is shown in its **Documentation** tab.

9.2 Reading the result

`execute_call` returns `True` and does not raise when the API answers with a failure: the result is then read on the endpoint.

Field	Content
`last_response_success`	True if the HTTP status is below 400
`last_response_status`	HTTP status, 0 without a response
`last_response_data`	Data received, as text (indented JSON for a JSON response)
`last_response_error`	Error message, empty on success
`last_response_time`	Duration in milliseconds

```
connector.execute_call({'name': partner.name})
if connector.last_response_success:
    data = json.loads(connector.last_response_data)
```

`execute_call` raises an error if the endpoint is archived, or if the record passed is not of the source model. An invalid body, an empty required value or a missing record for variables reading a field are recorded as a failure, in the last response and the log, without raising.

10 Monitoring

10.1 Call log

Monitoring > Call Logs lists every call: status, date, endpoint, success, HTTP code, duration and response size. Filters keep only successes or failures, and group by endpoint, status, code or day.

The details of a call show the data passed at execution time, the data received and, for a failure, the error message.

The **Retry** button runs the call again with the same execution data. The new call is added to the log.

10.2 Retention

The scheduled action **API Connector: Cleanup Old Logs** runs every day and deletes entries older than 30 days. The duration is set in the code of the related server action (`model.cleanup_old_logs(days=30)`).

11 Export and Import

11.1 Export

The **Export Config** button of an endpoint or a collection downloads a JSON file. A collection export contains its endpoints. An endpoint also exports its variables with their Odoo field, its source model and its body lines; on import, that model must exist in the database.

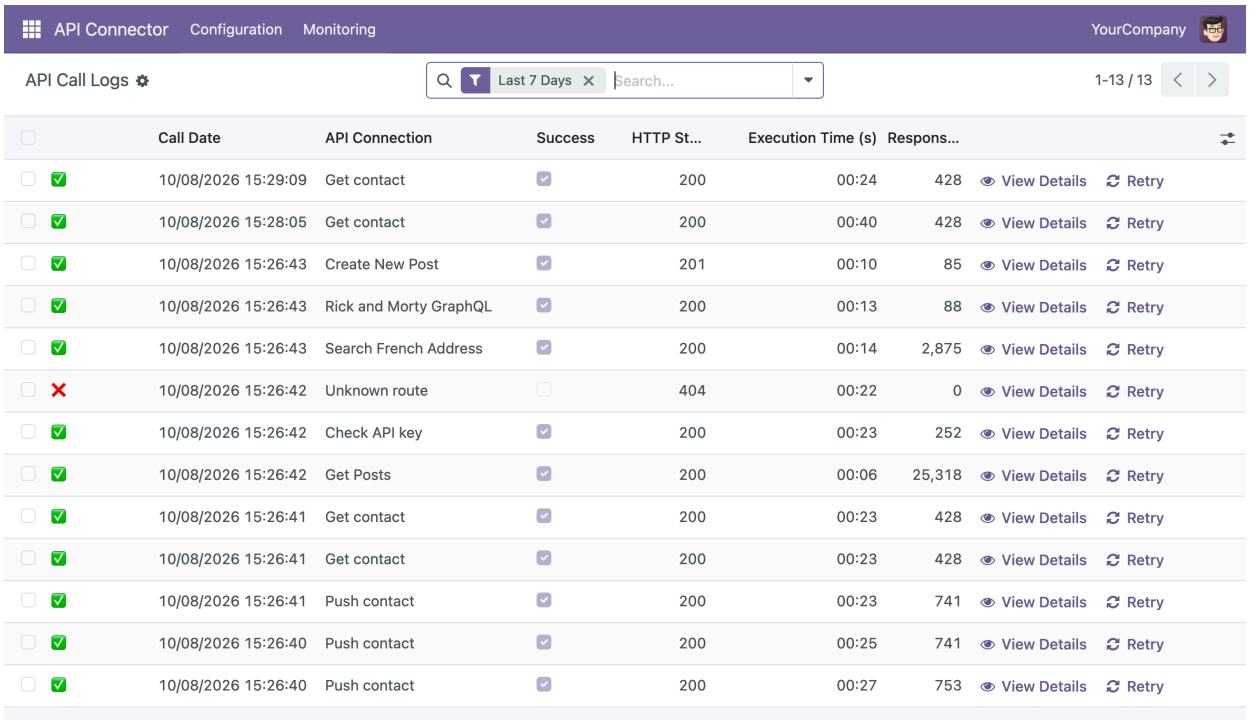


Figure 13: Call log: the unknown route returns a 404 and counts as a failure.

> **Warning:** the exported file contains the credentials entered: passwords, tokens, API keys and OAuth secrets. Store and share it like a password.

11.2 Import

Configuration > Import Configuration opens the import wizard.

Configuration Type	Effect
Auto-detect	Recognises the type from the file content.

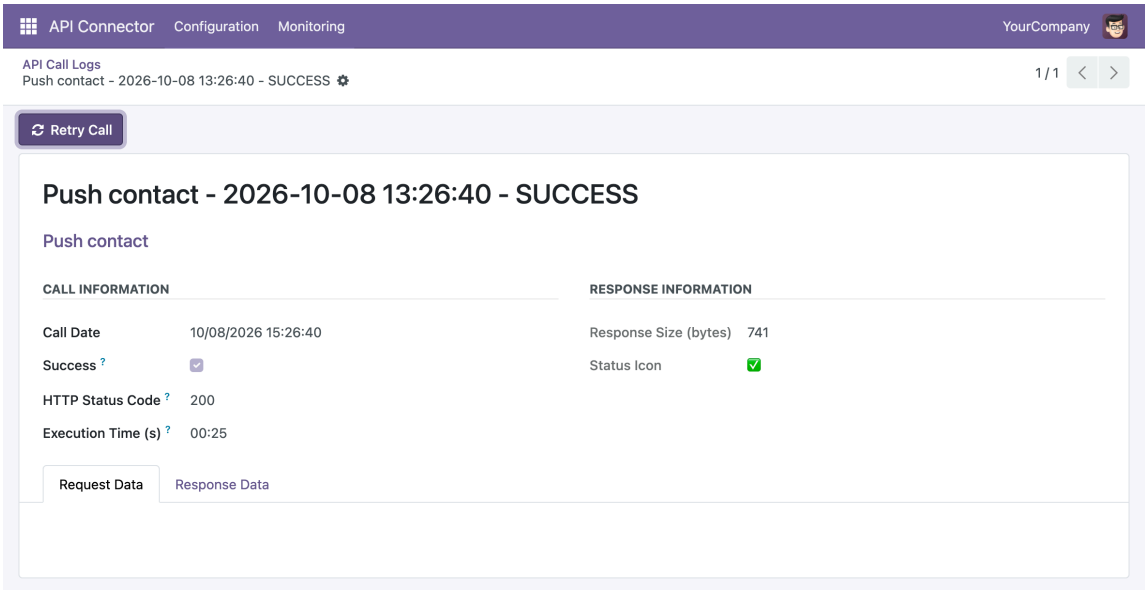


Figure 14: Details of a call made by the scheduled action: the values sent for this contact.

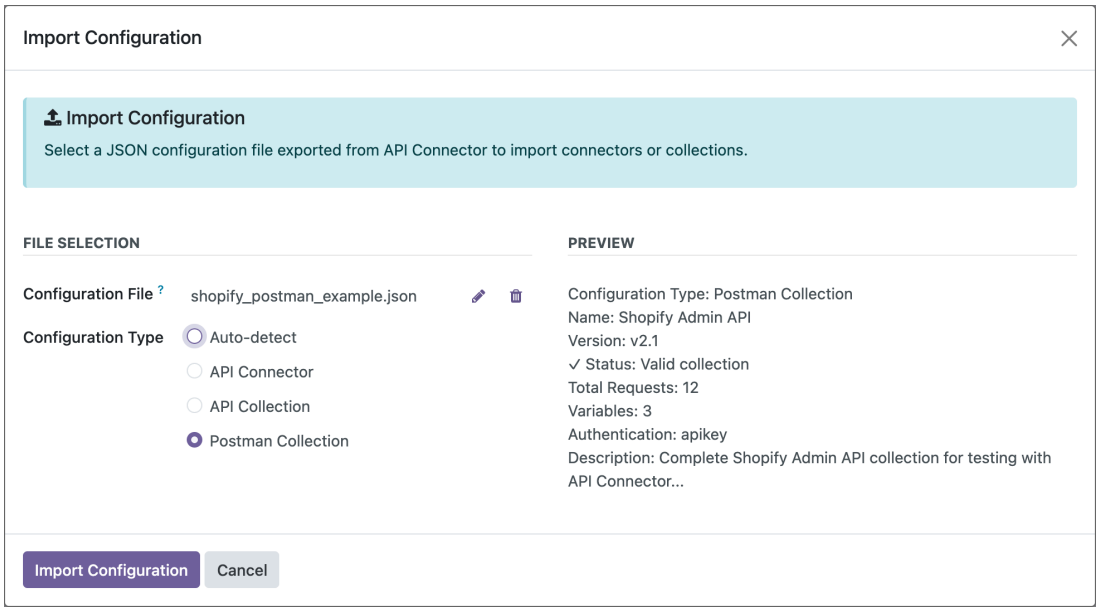


Figure 15: Import wizard: the Postman collection is recognised and summed up before import.

API Connector	Creates an endpoint, in the chosen **Target Collection** or, failing that, in the collection named as in the export if it exists.
API Collection	Creates the collection and its endpoints.
Postman Collection	Converts a Postman collection.

Import always creates new records: it does not update an existing configuration.

11.3 Postman collections

Accepted versions: v2.0, v2.1 and v2.2 (Postman **Collection v2.1** export).

Postman element	Result in Odoo
Collection	A collection, with its variables and authentication.
Folders	Flattened: the folder name prefixes the request name (`Products - Get All Products`).
Request	A REST endpoint, with its method, path and headers.
`{{variable}}`	Converted to `{variable}` in the path and the body.
Query parameters and path variables	Endpoint variables. A path variable gets the temporary value `{var_name}`.
Raw body	JSON when valid, raw text otherwise.
GraphQL body	JSON body `{"query": ..., "variables": ...}`.
Basic, bearer, apikey authentication	Imported with its values.

The wizard refuses a collection containing scripts (pre-request or test), NTLM, Hawk, AWS Signature or EdgeGrid authentication, a file body, certificates or proxy settings. A request whose body is `form-data` or `x-www-form-urlencoded` makes the import fail: remove it from the collection before exporting, then create it in Odoo.

After the import:

1. fill in the collection's base URL, which the import leaves empty;

2. replace the temporary `{var_name}` values of the endpoint variables, which win over the collection variables of the same name;
3. enter the real credentials: a token written `{{variable}}` in Postman is imported as is.

12 Access Rights

Group	Rights
Internal user	Read, create, edit and delete collections and endpoints. Read the log.
Administration (Settings)	All rights, the log included.

The module also creates the **API Connector User** and **API Connector Manager** groups. The rights above do not depend on these groups.

Credentials (passwords, tokens, keys, secrets) are stored in the database and masked on screen. Any internal user who opens an endpoint can execute it.

13 Technical Reference

13.1 Models

Model	Purpose
`api.collection`	Collection and shared settings
`api.connector`	Endpoint, execution and last response
`api.variable`	Variable of a collection or an endpoint, with its optional Odoo field
`api.body.line`	Line of a built body
`api.call.log`	One line per call
`api.request.preview`	Request preview (transient model, with its `api.request.preview.line` lines)
`import.config.wizard`	Import wizard
`postman.parser`	Postman collection conversion (abstract model)

13.2 Useful methods

Model	Method	Effect
`api.connector`	`execute_call(dynamic_data=None, record=None)`	Runs the call with the optional values `dynamic_data` and record `record` of the source model, logs it, updates the statistics and the last response. Returns `True`.
`api.connector`	`add_body_field_lines(field_paths)`	For each field path (`partner_id.name`), creates the route variable reading it and the body line using it.
`api.connector`	`get_oauth_token()`	Requests an OAuth 2.0 token (client credentials).
`api.collection`	`get_oauth_token()`	Same at collection level.
`api.connector`, `api.collection`	`export_config()`	Creates the JSON attachment of the export.

`api.call.log`	`cleanup_old_logs(days=30)`	Deletes entries older than `days` days.
`api.call.log`	`get_statistics(connector_id, days)`	For one endpoint (or all) over `days` days (7 by default), returns a dictionary: total, successes, failures, success rate, average duration.

13.3 Variables as JSON

The fields `collection_variables` (collection) and `endpoint_variables` (endpoint) give the variables as a JSON object `{name: value}`. Writing these fields rebuilds the variable lines: code or data files that used them keep working.

13.4 Deleting a critical endpoint

Deleting an endpoint flagged **System Critical** is refused. From code, it goes through with the `force_delete_critical` context:
`connector.with_context(force_delete_critical=True).unlink()`.